Identifying Fake News with NLP

Group 3: Michael Ezeanioma, Elizabeth Jones, Xiaolong Liu, Andrew Millman, John Sebesta

Project Overview

Proposal

The spread of misinformation is a growing concern in the digital age, impacting public perception and decision-making. Our project aims to develop a **machine learning model** that can classify news articles as either **real or fake** using **Natural Language Processing (NLP)** techniques. By analyzing text patterns, word distributions, and sentiment, we will determine the authenticity of news content.

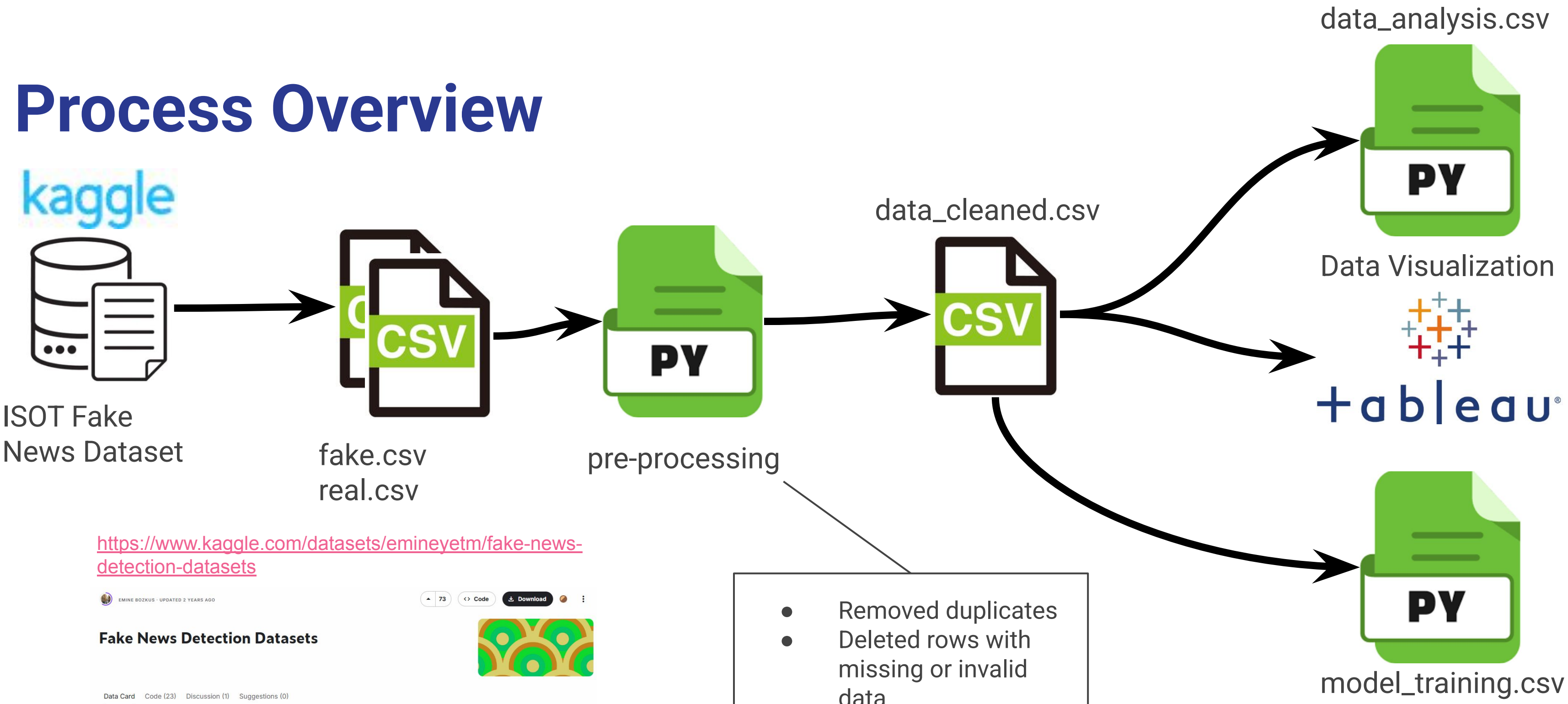
Why Fake News Detection Matters

- Fake news influences **public opinion, elections, and social behavior**
- Traditional detection methods are **manual and inefficient**

Project Goal

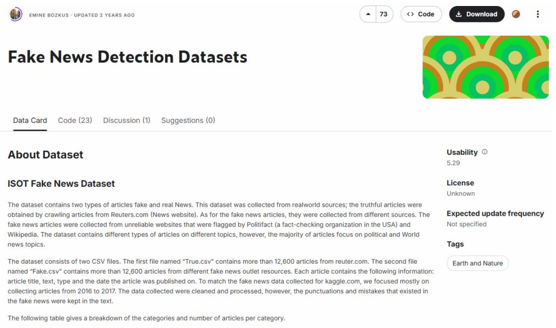
- Use **Machine Learning & NLP** to classify **real vs. fake news**
- Improve accuracy with **GRU-based text analysis**

Process Overview



- Removed duplicates
- Deleted rows with missing or invalid data

https://www.kaggle.com/datasets/emineyetm/fake-news-detection-datasets



EMINE BOZKUS · UPDATED 2 YEARS AGO

73 Code Download

Fake News Detection Datasets

Data Card Code (23) Discussion (1) Suggestions (0)

About Dataset

ISOT Fake News Dataset

The dataset contains two types of articles fake and real News. This dataset was collected from realworld sources; the truthful articles were obtained by crawling articles from Reuters.com (News website). As for the fake news articles, they were collected from different sources. The fake news articles were collected from unreliable websites that were flagged by Politifact (a fact-checking organization in the USA) and Wikipedia. The dataset contains different types of articles on different topics, however, the majority of articles focus on political and World news topics.

The dataset consists of two CSV files. The first file named "True.csv" contains more than 12,600 articles from reuter.com. The second file named "Fake.csv" contains more than 12,600 articles from different fake news outlet resources. Each article contains the following information: article title, text, type and the date the article was published on. To match the fake news data collected for kaggle.com, we focused mostly on collecting articles from 2016 to 2017. The data collected were cleaned and processed, however, the punctuations and mistakes that existed in the fake news were kept in the text.

The following table gives a breakdown of the categories and number of articles per category.

Usability
5.29

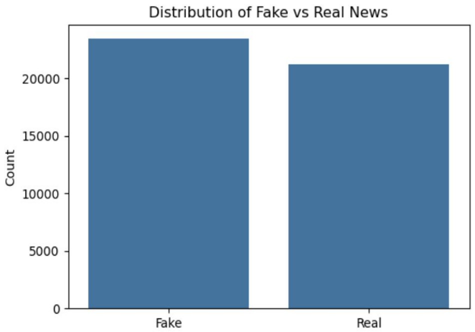
License
Unknown

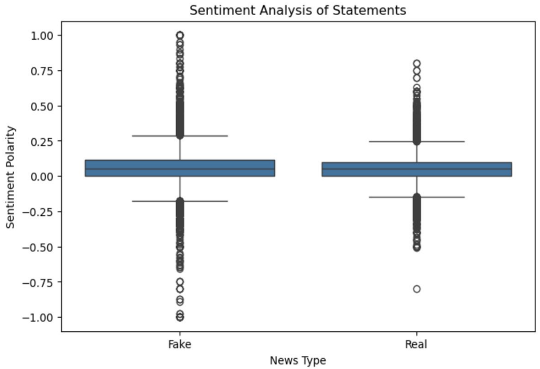
Expected update frequency
Not specified

Tags
Earth and Nature

Data Overview

- **Total Records:** ~38,000 rows of date/labeled statements
- **Features Used:**
  - **Statement (Text Input)** → Core feature
  - **Subject** → Contextual features
  - **Label (Fake or Real)** → Supervised classification





Preprocessing Steps

- Handled **missing values**
- Converted labels into **binary classification (Fake = 0, Real = 1)**
- Tokenized text using **GRU Tokenizer**

https://public.tableau.com/app/profile/andrew.millman/viz/Project4DataAnalysis_17413167569810/DataAnalysis?publish=yes

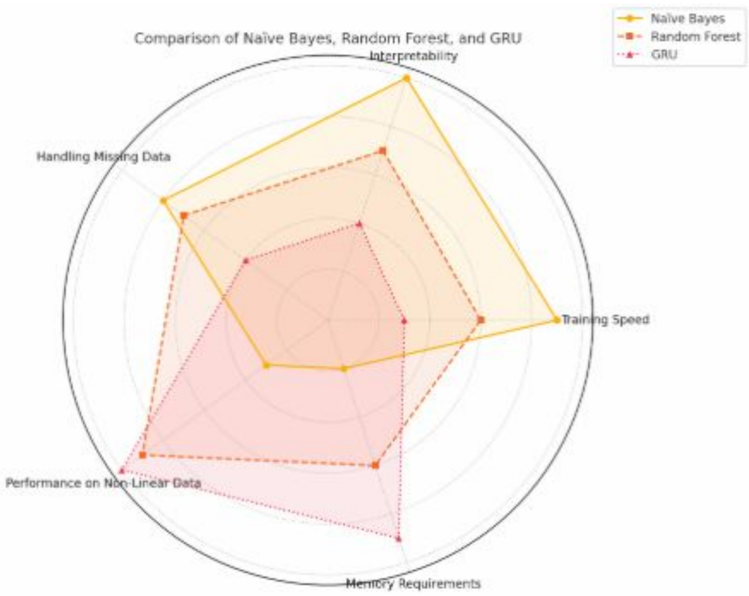
Machine Learning Process

Baseline Models:

- Logistic Regression (Accuracy: ~62%)
- Naïve Bayes & Random Forest (Lower Accuracy)

Final Model: GRU

- Captures **context better** than traditional models
- Uses **Tokenization & Embeddings**



Results & Performance

Final Model Performance

- Accuracy: **~98%**
- Precision & Recall: **Balanced results**

Confusion Matrix & Insights

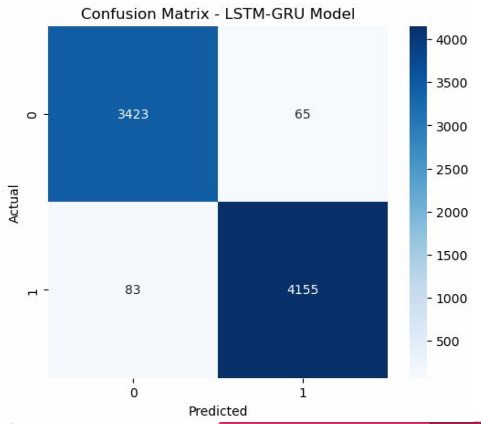
- Correctly classified **most real and fake news**
- Some misclassifications due to **ambiguous language**

Feature Importance & Trends

- **Certain words & phrases** strongly correlated with fake news
- **Party Affiliation & Speaker Trends** identified

LSTM-GRU Model Accuracy: 0.9808

	precision	recall	f1-score	support
0	0.98	0.98	0.98	3488
1	0.98	0.98	0.98	4238
accuracy			0.98	7726
macro avg	0.98	0.98	0.98	7726
weighted avg	0.98	0.98	0.98	7726



Challenges & Future Improvements

Challenges Faced

- Size and balance of the data set
- Duplicates after we thought our data was cleaned thoroughly

Future Improvements

- Use **Larger Transformer Models (e.g., RoBERTa, GPT)**
- Include **Sentiment Analysis** for deeper insights
- Apply to **real-time fake news detection**

Conclusion & Q&A

Final Summary

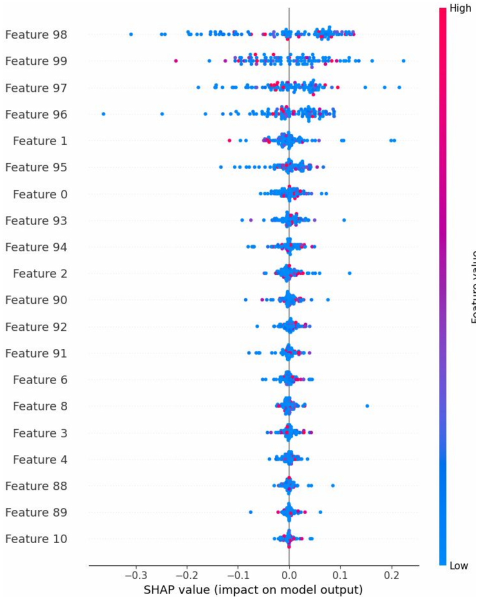
- Machine Learning can **effectively detect fake news**
- GRU significantly **outperformed traditional models**

Questions?

Thank you!

Validating the Model

- Ensured adequate shuffling of the data
- Checked for data leakage




```python
# Inspect Word Frequencies Across Classes:
for word in ['told', 'obama', 'white', 'into', 'campaign', 'two', 'do', 'against', 'like', 'election', 'because', 'them', 'last']:
    print(f"'{word}' appears {df[df['real'] == 1]['text'].str.contains(word, case=False).sum()} times in REAL news")
    print(f"'{word}' appears {df[df['real'] == 0]['text'].str.contains(word, case=False).sum()} times in FAKE news")
```

```
'told' appears 9146 times in REAL news
'told' appears 4408 times in FAKE news
'obama' appears 4119 times in REAL news
'obama' appears 4801 times in FAKE news
'white' appears 4802 times in REAL news
'white' appears 4790 times in FAKE news
'into' appears 7843 times in REAL news
'into' appears 8680 times in FAKE news
'campaign' appears 5322 times in REAL news
'campaign' appears 4378 times in FAKE news
'two' appears 8313 times in REAL news
'two' appears 5135 times in FAKE news
'do' appears 17457 times in REAL news
'do' appears 15759 times in FAKE news
'against' appears 6704 times in REAL news
'against' appears 4939 times in FAKE news
'like' appears 5858 times in REAL news
'like' appears 9431 times in FAKE news
'election' appears 6258 times in REAL news
'election' appears 3726 times in FAKE news
'because' appears 4537 times in REAL news
'because' appears 7239 times in FAKE news
'them' appears 5515 times in REAL news
'them' appears 7374 times in FAKE news
'last' appears 8605 times in REAL news
'last' appears 4970 times in FAKE news
```

```python
# Load the tokenizer
tokenizer = joblib.load("tokenizer.pkl")  # Ensure the correct path

# Retrieve the word index from the tokenizer
word_index = tokenizer.word_index
index_word = {v: k for k, v in word_index.items()}  # Reverse mapping

# Replace this with actual SHAP values ranking if available
important_token_indices = list(range(87, 100))  # Example based on SHAP summary plot

# Map token indices back to actual words
important_words = [index_word.get(idx, "<UNK>") for idx in important_token_indices]

print(f"Top important words: {important_words}")
```

```
Top important words: ['told', 'obama', 'white', 'into', 'campaign', 'two', 'do', 'against', 'like', 'election', 'because', 'them', 'last']
```